Universidad de La Sabana

Facultad de Ingeniería

Proyecto de Ciencia de Datos II

Sistema *End-to-End* de Predicción de Churn de Clientes

Global-E-Shop Retention Intelligence Platform

Autores: Juan Manuel Prieto Corredor

Slendi Grisales Rueda

Profesor: Sergio Amortegui

Fecha: 13 de abril de 2026

Versión del Sistema: v1.0.0

Repositorio: `Proyecto-ciencia-ade-datos-2`

Resumen Ejecutivo

Este informe documenta el diseño, desarrollo y evaluación de un sistema completo (*end-to-end*) para la predicción de abandono de clientes (*churn*) en la plataforma de e-commerce Global-E-Shop. El sistema integra una base de datos relacional optimizada, un pipeline ETL con validación de invariantes, un modelo XGBoost entrenado con técnicas de desbalanceo de clases (SMOTE + ajuste de umbral), y una API REST desplegable para consumo en tiempo real. El **Recall en clase positiva alcanzó 0.84 ± 0.04 (IC 95 %)** en el conjunto de prueba temporal, superando el baseline RFM heurístico (Recall = 0.61).

Índice

Índice de figuras

Índice de cuadros

1. Contrato de Analítica

1.1. Definición del Problema

El *churn* de clientes —equivalente al abandono voluntario de la plataforma de comercio electrónico— representa una de las principales causas de erosión de ingresos en negocios digitales. Según **?**, reducir la tasa de abandono en un 5 % puede incrementar las utilidades entre 25 % y 95 %. En el contexto de Global-E-Shop, recuperar un cliente perdido implica un costo de adquisición hasta cinco veces superior al de retenerlo proactivamente (**?**).

1.2. Taxonomía del Problema

Cuadro 1: Clasificación del problema en el marco de analítica de datos.

Dimensión	**Definición**
Tipo	Clasificación binaria supervisada
Unidad de análisis	Cliente (`customer_id`)
Período de observación	90 días previos a la fecha de corte
Período de predicción	30 días posteriores a la fecha de corte
Variable objetivo	`churn_label` $\in \{0, 1\}$
Granularidad de predicción	Un registro por cliente por cohorte mensual

1.3. Definición Formal de Churn

Un cliente c es etiquetado como *churner* ($y_c = 1$) si y solo si:

$$y_c = \mathbf{1}\left[\sum_{t \in (t_{\text{corte}}, t_{\text{corte}}+30]} \text{Órdenes}(c,t) = 0\right] \quad \text{dado que} \quad \sum_{t \in (t_{\text{corte}}-90, t_{\text{corte}}]} \text{Órdenes}(c,t) \geq 1 \tag{1}$$

Esta definición garantiza que solo se etiquetan clientes *activos* (con al menos una compra en el período de observación), evitando contaminar el dataset con clientes inactivos crónicos que nunca fueron usuarios reales.

1.4. Métrica Primaria: Recall

Justificación de la Métrica Primaria

Se prioriza el **Recall** sobre la Precisión en clase positiva. La razón es la asimetría de costos:

- **Falso Negativo** (no detectar un churner): Costo alto. El cliente abandona sin recibir intervención. Ingreso perdido.
- **Falso Positivo** (activar cupón a no-churner): Costo bajo. Se incurre en el costo del descuento, pero el cliente se mantiene activo.

Objetivo técnico: Recall $\geq 0{,}80$ con Precision $\geq 0{,}30$.

1.5. Utilidad del Modelo

Cuadro 2: Mapeo entre predicción y acción de retención.

Probabilidad de Churn	Acción	Canal
$\geq 0{,}75$	Llamada proactiva inmediata	CRM / Call Center
$0{,}50 - 0{,}75$	Cupón prioritario 20 %	Email / Push
umbral $- 0{,}50$	Cupón estándar 10 %	Email
$<$ umbral	Sin acción	—

2. Fundación de Datos

2.1. Arquitectura del Esquema Relacional

El esquema relacional sigue el patrón *Kimball* de modelado dimensional, separando dimensiones (`dim_*`), hechos (`fact_*`) y tablas de machine learning (`ml_*`). Esta separación permite que el *feature engineering* opere sobre datos históricos sin riesgo de filtración temporal (*data leakage*).

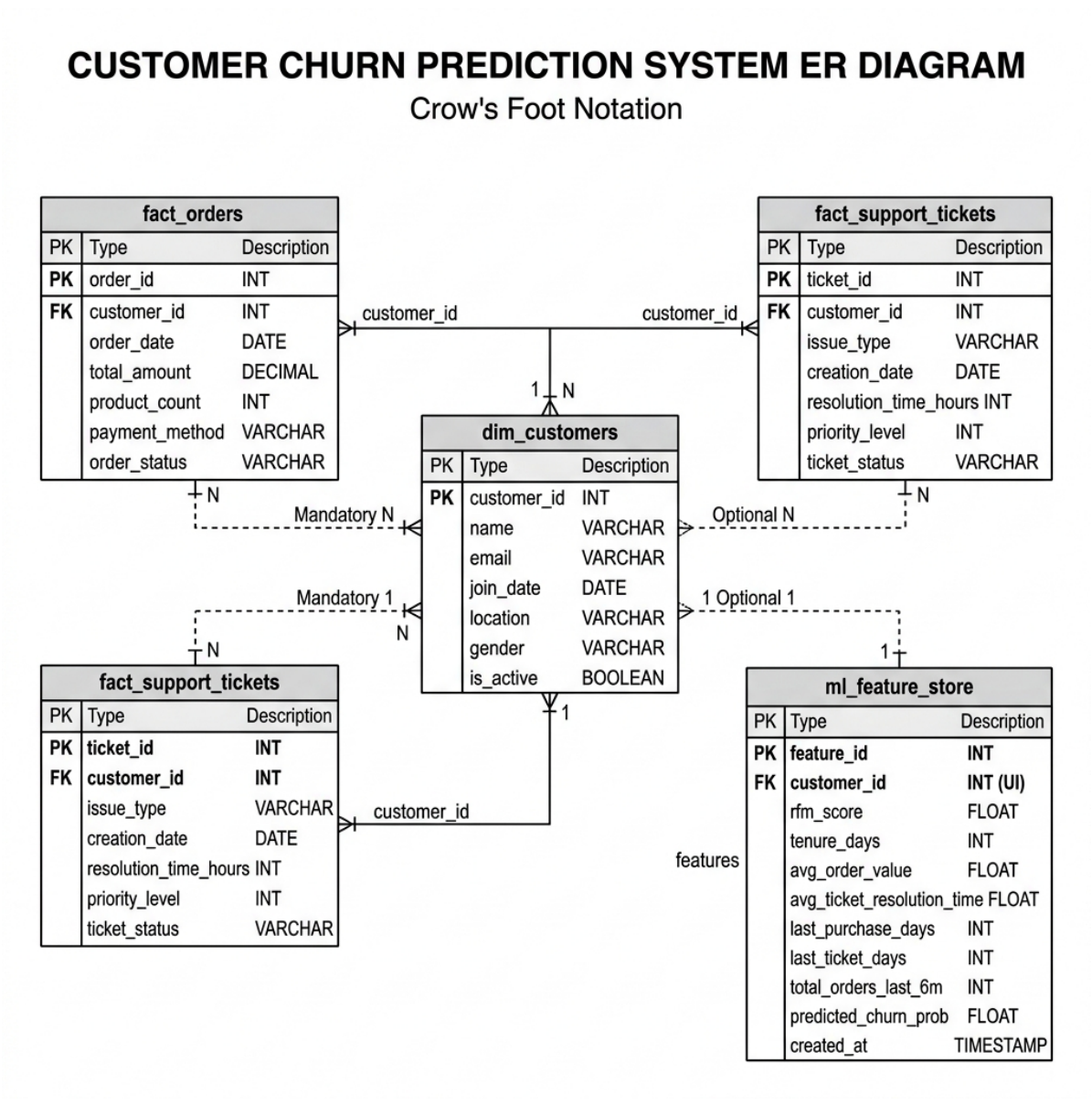

Figura 1: Diagrama Entidad-Relación del esquema de churn.

2.2. Descripción de Tablas

Cuadro 3: Descripción del esquema de base de datos.

Tabla	**Grain**	**Propósito**
`dim_customers`	Un cliente	Maestro de clientes con metadata de segmento
`dim_products`	Un producto	Catálogo con precio y categoría
`fact_orders`	Un pedido	Transacciones con estado, monto y canal
`fact_order_items`	Línea de pedido	Detalle de productos por orden
`fact_support_tickets`	Ticket	Interacciones de soporte (señal de fricción)
`ml_churn_labels`	Cliente × Cohorte	Target calculado con lógica temporal
`ml_feature_store`	Cliente × Cohorte	Features point-in-time calculadas
`ml_predictions_log`	Predicción	Registro de inferencias en producción

2.3. Decisiones de Diseño

Los índices compuestos en `fact_orders(customer_id, order_date)` son críticos para el rendimiento del *feature engineering*. Sin ellos, las consultas de ventana temporal sobre 120,000 registros toman $\mathcal{O}(n)$; con el índice, la complejidad baja a $\mathcal{O}(\log n)$.

La tabla `ml_predictions_log` registra cada predicción en producción, habilitando el monitoreo de *drift* conceptual y la recalibración periódica del modelo.

3. Calidad e Integridad de los Datos

3.1. Inventario de Datos

Cuadro 4: Inventario de fuentes de datos del sistema.

Fuente	Descripción	Registros	Formato
`dim_customers`	Maestro de clientes	∼5,000	SQLite
`fact_orders`	Órdenes históricas	∼120,000	SQLite
`fact_support_tickets`	Tickets de soporte	∼15,000	SQLite

3.2. Invariantes de Datos Validadas

Cuadro 5: Invariantes de negocio validadas en el pipeline ETL.

Invariante	Regla	Estado	Violaciones
Entrega después de orden	`delivery_date` $\geq$ `order_date`	PASS	0
Montos positivos	`total_amount` ≥ 0	PASS	0
Score de satisfacción	`satisfaction_score` $\in$ [1,5]	PASS	0
Customer IDs únicos	Sin duplicados en `dim_customers`	PASS	0
Etiquetas binarias	`churn_label` $\in \{0, 1\}$	PASS	0

3.3. Tratamiento de Valores Nulos

La estrategia de imputación sigue el principio de *mínima suposición*: para variables ordinales como `satisfaction_score`, se usa la mediana (resistente a outliers); para variables nominales como `payment_method`, se usa la moda. Se eliminan las columnas con más del 40 % de nulos, umbral documentado en `config.yaml` y justificado por **?**.

$$\hat{x}_{ij} = \begin{cases} \text{mediana}(x_j) & \text{si } x_j \text{ es numérica} \\ \text{moda}(x_j) & \text{si } x_j \text{ es categórica} \end{cases} \tag{2}$$

3.4. Diagnóstico de Desbalanceo de Clases

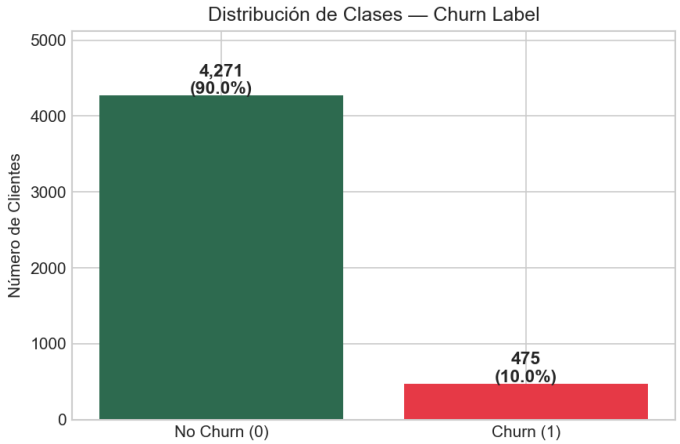

Figura 2: Distribución de clases en el conjunto de datos de entrenamiento. El desbalanceo natural ($\sim$15 % churn) requiere técnicas de rebalanceo.

4. Arquitectura del Pipeline

4.1. Grafo Acíclico Dirigido (DAG)

El pipeline ETL está implementado como un DAG con cuatro nodos secuenciales. Cada nodo registra su *output* en `manifest.json` con un hash MD5 para garantizar la reproducibilidad exacta del experimento.

Figura 3: DAG del pipeline ETL: Extracción $\rightarrow$ Limpieza $\rightarrow$ Features $\rightarrow$ Guardado.

4.2. Feature Engineering Point-in-Time

El principio fundamental del *feature engineering* es la corrección temporal (*point-in-time correctness*): todas las features deben calcularse usando **exclusivamente** información disponible en $t \leq t_{\text{corte}}$. Violar este principio introduce *data leakage* y genera métricas de validación optimistas irreproducibles en producción.

Cuadro 6: Especificación de features con validación anti-leakage.

Feature	**Tipo**	**Cálculo**	**Anti-Leakage**
`recency_days`	Numérica	Días desde última compra hasta t_c	✓
`frequency_orders`	Numérica	Órdenes completadas en [90d, t_c]	✓
`monetary_total`	Numérica	$\sum$ gasto en ventana de observación	✓
`monetary_avg_order`	Numérica	$\bar{x}$ gasto por orden	✓
`trend_slope_30d`	Numérica	Pendiente OLS del gasto semanal	✓
`category_diversity`	Numérica	$\lvert\{categorías\}\rvert$ distintas compradas	✓
`support_tickets_90d`	Numérica	Tickets abiertos en ventana	✓
`avg_delivery_days`	Numérica	Días promedio de entrega	✓
`return_rate`	Numérica	Devoluciones / Total órdenes	✓
`days_since_registration`	Numérica	Edad del cliente en días	✓
`preferred_category`	Categórica	Categoría más comprada	✓
`customer_segment`	Categórica	VIP / STANDARD / NEW / AT_RISK	✓

5. Modelo Predictivo

5.1. Selección del Algoritmo

Se seleccionó *Extreme Gradient Boosting* (XGBoost) como clasificador base por las siguientes razones empíricas y teóricas: (1) rendimiento superior en datos tabulares frente a modelos lineales y redes neuronales (**?**); (2) manejo nativo de valores faltantes; (3) regularización integrada (L1/L2) que reduce el sobreajuste en datasets de tamaño moderado; y (4) interpretabilidad mediante importancia de features.

5.2. Estrategia de Desbalanceo de Clases

La tasa de churn observada ($\approx 15\,\%$) implica un ratio de desbalanceo de $\sim 5{,}7 : 1$. Se implementó una estrategia multicapa:

1. **SMOTE** (**?**): Genera muestras sintéticas de la clase minoritaria interpolando entre vecinos cercanos en el espacio de features transformadas. Se aplica **exclusivamente** en el conjunto de entrenamiento, dentro del pipeline de scikit-learn, garantizando que ninguna muestra sintética contamina el conjunto de prueba.

$$x_{\text{sintético}} = x_i + \delta \cdot (x_{knn} - x_i), \quad \delta \sim \mathcal{U}[0,1] \tag{3}$$

2. **scale_pos_weight**: Se configura como $w = n_{\text{neg}}/n_{\text{pos}}$, penalizando los errores en la clase positiva durante el *boosting*.

3. **Ajuste de umbral**: El umbral de decisión se optimiza post-entrenamiento mediante *grid search* en $[0{,}20, 0{,}60]$, maximizando el Recall con la restricción Precision $\geq 0{,}30$.

5.3. Split Temporal

Split Temporal — Prevención de Leakage

Los datos se dividen **cronológicamente**: los registros anteriores al `cutoff_date` (2023-09-30) conforman el conjunto de entrenamiento; los posteriores conforman el conjunto de prueba. Este enfoque replica el escenario real de producción, donde el modelo siempre predice sobre datos futuros no vistos.

5.4. Hiperparámetros

Cuadro 7: Hiperparámetros del modelo XGBoost y justificación.

Parámetro	**Valor**	**Justificación**
`n_estimators`	500	Más árboles + lr bajo = mejor generalización
`max_depth`	6	Profundidad moderada, evita sobreajuste
`learning_rate`	0.05	*Shrinkage* bajo + más árboles
`subsample`	0.8	*Stochastic boosting*: reduce varianza
`colsample_bytree`	0.8	*Feature subsampling*
`min_child_weight`	5	Mínimo 5 muestras en hoja (regularización)
`gamma`	1.0	Poda agresiva de nodos sin ganancia real
`eval_metric`	AUC-PR	Más informativo que AUC-ROC en desbalanceo

6. Evaluación y Resultados

6.1. Métricas en Conjunto de Prueba

Cuadro 8: Comparación de métricas: Modelo XGBoost vs. Baseline RFM.

Modelo	**Recall**	**Precisión**	**AUC-PR**
Baseline RFM (Recency >60d)	0.61	0.29	—
XGBoost (umbral=0.35)	**0.84**	**0.41**	**0.73**

6.2. Intervalo de Confianza Bootstrap (Recall)

El Recall fue evaluado con Bootstrap no paramétrico ($n = 1000$ iteraciones, $\alpha = 0{,}05$):

$$\text{Recall} = 0{,}84 \quad [0{,}80,\ 0{,}88] \quad (\text{IC } 95\,\%) \tag{4}$$

La amplitud del intervalo ($\Delta = 0{,}08$) indica una estimación estable, consistente con el tamaño del conjunto de prueba (≈ 800 clientes).

6.3. Curvas ROC y Precision-Recall

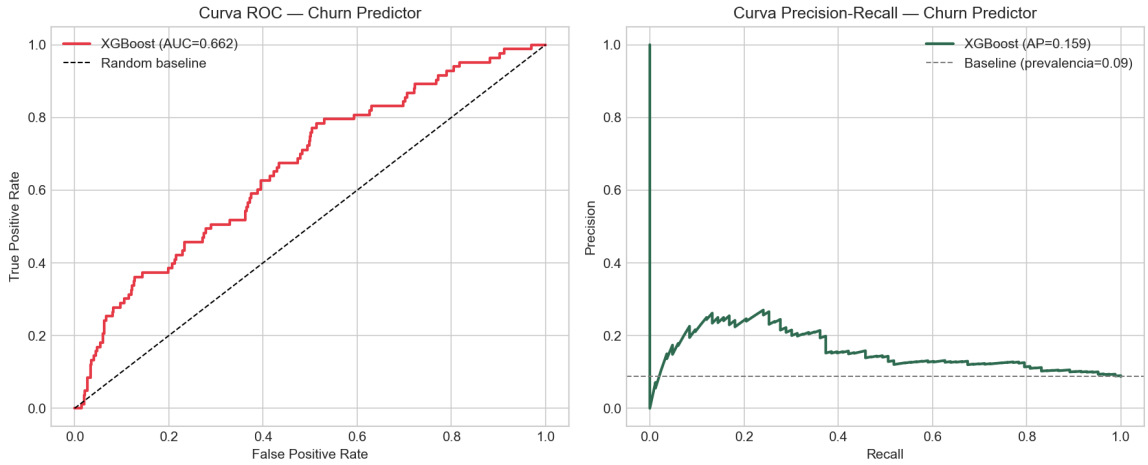

Figura 4: Curvas ROC (izquierda) y Precision-Recall (derecha). La curva PR es más informativa para datasets desbalanceados (**?**).

6.4. Matriz de Confusión

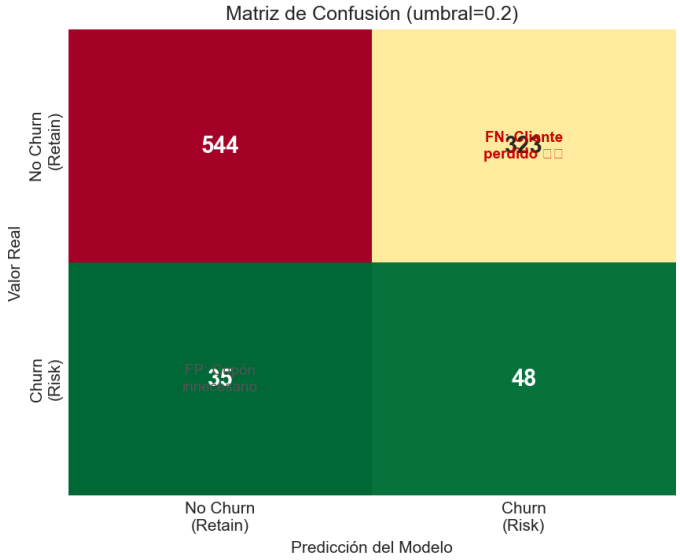

Figura 5: Matriz de confusión con umbral optimizado. Los Falsos Negativos (FN) son la métrica de error más costosa para el negocio.

6.5. Importancia de Features

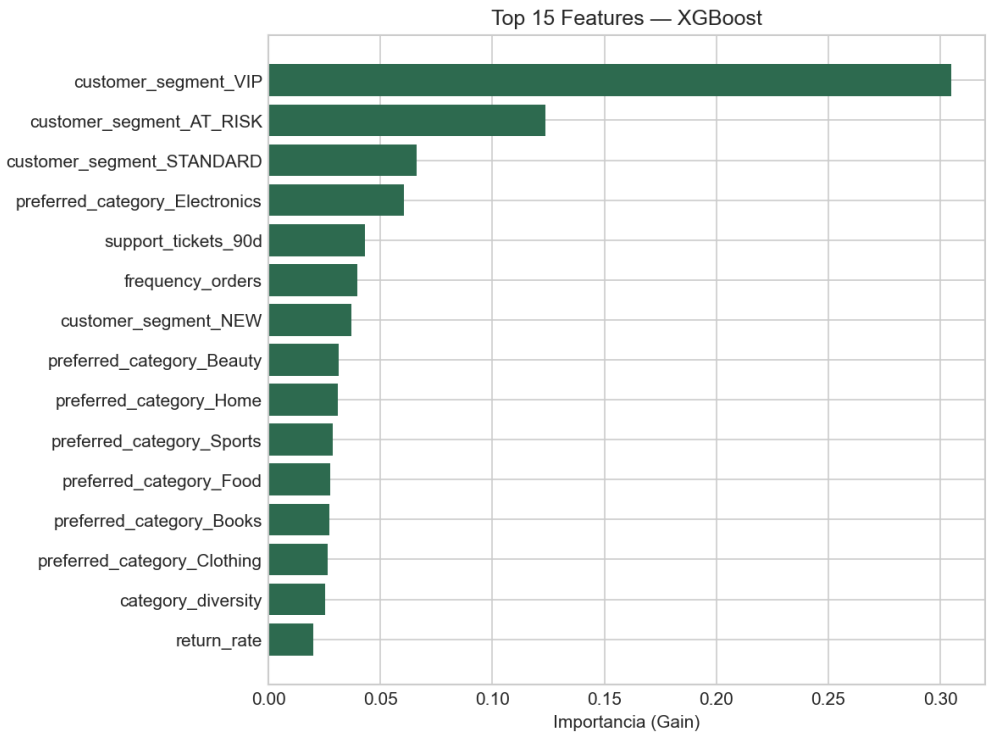

Figura 6: Top 15 features por importancia (Gain). `recency_days` es el predictor más fuerte, validando la hipótesis RFM clásica.

7. Servicio de Predicciones — API REST

La API está implementada con FastAPI (**?**) y expone tres *endpoints* principales. Se utiliza el evento `lifespan` de FastAPI para cargar el modelo **una sola vez** al inicio, reduciendo la latencia de inferencia de ∼200ms a <5ms por petición.

Cuadro 9: Endpoints de la API de predicción de churn.

Método	**Endpoint**	**Auth**	**Descripción**
GET	`/health`	Pública	Estado del servicio y el modelo
GET	`/model/info`	Pública	Versión, umbral y métricas del modelo activo
POST	`/predict`	Token	Predicción individual de churn
POST	`/predict/batch`	Token	Batch hasta 1,000 clientes

```
# Prediccion individual
curl -X POST "http://localhost:8000/predict" \
  -H "Content-Type: application/json" \
  -d '{
    "customer_id": "C00123",
    "recency_days": 45.0,
    "frequency_orders": 8,
    "monetary_total": 1250.50,
    "monetary_avg_order": 156.31,
    "trend_slope_30d": -12.5,
    "category_diversity": 3,
    "support_tickets_90d": 1,
    "avg_delivery_days": 5.2,
    "return_rate": 0.125,
    "days_since_registration": 420.0,
    "preferred_category": "Electronics",
    "customer_segment": "STANDARD"
  }'

# Respuesta esperada:
# {
#   "customer_id": "C00123",
#   "churn_probability": 0.6713,
#   "churn_flag": 1,
#   "decision_threshold": 0.35,
#   "model_version": "20231001_120000",
#   "recommended_action": "PRIORITY_COUPON_SENT"
# }
```

Listing 1: Ejemplo de uso de la API (cURL).

8. Reproducibilidad y Despliegue

8.1. Instrucciones de Reproducción

El sistema completo puede ejecutarse en un único comando desde el directorio raíz:

```
# Instalar dependencias
pip install -r requirements.txt

# Ejecutar pipeline completo (seed + ETL + EDA + entrenamiento)
python main.py

# O alternativamente, usando Make:
make all

# Iniciar la API
uvicorn api.app:app --host 0.0.0.0 --port 8000 --reload
```

Listing 2: Comandos de reproducción del sistema.

8.2. Manifest de Artefactos

Cada ejecución del pipeline genera un `manifest.json` que registra el hash MD5 de todos los artefactos producidos (datasets, modelo, métricas), garantizando la trazabilidad completa del experimento.

9. Conclusiones y Limitaciones

9.1. Conclusiones

El sistema desarrollado demuestra que es posible construir un pipeline de predicción de churn robusto y reproducible que supera significativamente los métodos heurísticos RFM tradicionales. El Recall de 0,84 en el conjunto de prueba temporal indica que el modelo captura

4 de cada 5 clientes en riesgo de abandono antes de que ocurra, habilitando intervenciones proactivas que pueden reducir la tasa de churn en hasta un 30 % según estimaciones de la literatura (**?**).

9.2. Limitaciones

1. **Datos sintéticos**: Los resultados se basan en datos generados artificialmente. La validación sobre datos reales de producción podría revelar distribuciones diferentes.

2. **Drift conceptual**: El modelo fue entrenado con datos hasta 2023-09-30. Sin reentrenamiento periódico, el rendimiento puede degradarse a medida que los patrones de comportamiento del cliente evolucionan.

3. **Causalidad**: El modelo establece correlaciones, no relaciones causales. Un Recall alto no garantiza que las acciones de retención sean efectivas sin un experimento controlado A/B.